SBA301 – LAB 06

STEP-BY-STEP GUIDE

SECURE REST SERVICES WITH SPRING BOOT USING JSON WEB TOKEN (JWT)

Spring Boot 3 • Spring Security • JWT • Spring Data JPA • SQL Server • Postman

CORE: Mục tiêu của tài liệu này là workbook có thể tự thực hiện, tự kiểm chứng và tự debug. Phần Core Lab bám yêu cầu gốc; phần Advanced Extension & Refactor là nội dung tự nghiên cứu có chủ đích, không bắt buộc để hoàn thành Lab baseline.

Thông tin	Giá trị
Lab	Lab 06 – Secure REST Services with Spring Boot using JWT
Baseline nghiệp vụ	Registration + Login + JWT Authentication + Protected User endpoints
Backend	Spring Boot 3 / Spring MVC / Spring Security / Spring Data JPA
Database	Microsoft SQL Server
Token library course baseline	JJWT 0.12.6 (theo tài liệu Lab gốc)
Verification	Postman + console/security logs + SQL Server + automated tests
Core endpoints	POST /auth/signup • POST /auth/login • GET /users/me • GET /users
Learning pattern	Build → Run → Test → Break → Diagnose → Fix → Refactor → Extend

1. TÓM TẮT NỘI DUNG VÀ YÊU CẦU

Lab 06 yêu cầu xây dựng một RESTful API có đăng ký và đăng nhập, dùng Spring Security để xác thực người dùng và JWT làm short-lived credential cho các request sau đăng nhập. User được lưu trong SQL Server thông qua Spring Data JPA. Sau khi login thành công, client gửi token ở Authorization header dạng Bearer để gọi protected endpoints.

SECURITY: JWT payload không được xem là dữ liệu bí mật. Signed JWT đảm bảo integrity/authenticity theo khóa ký; nó không tự động mã hóa payload.

1.1. Luồng hệ thống cần đạt

```
Client / Postman
   |
   | POST /auth/signup
   v
AuthenticationController -> AuthenticationService -> PasswordEncoder -> UserRepository -> SQL Server

Client / Postman
   |
   | POST /auth/login (email + password)
   v
AuthenticationController -> AuthenticationManager -> AuthenticationProvider
   -> UserDetailsService -> UserRepository
   -> PasswordEncoder.matches(...)
   -> JwtService.generateToken(...)
   <- JWT

Protected request
Authorization: Bearer <jwt>
   |
   v
SecurityFilterChain -> JwtAuthenticationFilter
   -> validate token -> load user -> SecurityContext
   -> Authorization rule
   -> /users/me or /users
```

1.2. Kết quả tối thiểu phải đạt

- Đăng ký User và password lưu dưới dạng hash, không lưu plaintext.
- Login đúng credential trả JWT + expiration metadata.
- Không token hoặc token lỗi không được truy cập protected endpoint.

- JWT hợp lệ truy cập được `/users/me`.
- Endpoint `/users` được bảo vệ theo policy đã chọn trong Core Lab.
- Có error response hợp lý cho authentication failure.
- Có Postman evidence và SQL evidence.
- Có ít nhất một automated security test cho happy path và negative path.

2. CÔNG NGHỆ, PHẦN MỀM VÀ ĐIỀU KIỆN CHUẨN BỊ

Thành phần	Baseline khuyến nghị	Kiểm tra
JDK	21 (course baseline)	java -version
IDE	IntelliJ IDEA	Project SDK = 21
Build	Maven	mvn -v
Spring Boot	3.x theo course project	pom.xml
Database	SQL Server	SSMS / SELECT 1
API client	Postman	request chạy được
JWT	JJWT 0.12.6 theo Lab source	dependency tree
Git	Checkpoint trước Security	git status

2.1. Pre-flight checklist

```
java -version
mvn -v
git status

# SQL Server smoke test trong SSMS
SELECT 1 AS ok;
```

- SQL Server service đang chạy.
- Biết username/password DB của máy mình nhưng không đưa credential thật vào Git.
- Port backend chưa bị chiếm; Lab gốc dùng `server.port=8005`.
- Có thể tạo database/schema cho User.
- Postman có Environment hoặc Collection variables để lưu `baseUrl` và `token`.

3. CẤU TRÚC THƯ MỤC MỤC TIÊU

```
src/main/java/<base-package>/
├─ configurations/
│  ├─ ApplicationConfiguration.java
│  ├─ JwtAuthenticationFilter.java
│  └─ SecurityConfiguration.java
├─ controllers/
│  ├─ AuthenticationController.java
│  └─ UserController.java
├─ dtos/
│  ├─ LoginUserDto.java
│  └─ RegisterUserDto.java
├─ entities/
│  └─ User.java
├─ exceptions/
│  └─ GlobalExceptionHandler.java
├─ repositories/
│  └─ UserRepository.java
├─ responses/
│  └─ LoginResponse.java
└─ services/
   ├─ AuthenticationService.java
   ├─ JwtService.java
   └─ UserService.java

src/main/resources/
└─ application.properties
```

NOTE: Cấu trúc này bám Lab gốc. Ở phần Refactor, sinh viên sẽ được yêu cầu tách security/auth domain rõ hơn và giảm coupling giữa entity với Spring Security interfaces.

4. TÓM TẮT TOÀN BỘ CÁC BƯỚC THỰC HIỆN

#	Activity/Step	Checkpoint
0	Chuẩn bị SQL Server / project baseline	DB + build OK
1	Tạo Spring Boot project + dependencies	App starts
2	Cấu hình datasource + JWT properties	DB connects
3	Tạo User entity/UserDetails + Repository	User table/repo OK
4	Tạo DTO/response	JSON contract rõ
5	Tạo PasswordEncoder/UserDetailsService/AuthProvider	Username/password auth works
6	Tạo JwtService	Generate/parse/validate token
7	Tạo JwtAuthenticationFilter	Bearer token -> SecurityContext
8	Tạo SecurityFilterChain	public/protected paths đúng
9	Tạo AuthenticationService/Controller	signup/login works
10	Tạo UserService/UserController	/users/me protected
11	Global error handling	401/other errors rõ
12	Postman end-to-end + negative tests	Core Lab Gate pass
13	Advanced / Extension / Refactor	self-study tracks

5. ACTIVITY 0 – CHUẨN BỊ SQL SERVER VÀ DATABASE

Step 0.1 – Tạo database

MỤC TIÊU: Có database riêng cho Lab 06.

SQL Server

```
IF DB_ID(N'OrchidJWTLab') IS NULL
    CREATE DATABASE OrchidJWTLab;
GO
USE OrchidJWTLab;
GO
SELECT DB_NAME() AS current_db;
```

PASS/FAIL: PASS nếu current_db = OrchidJWTLab.

Step 0.2 – Không tạo bảng users thủ công nếu dùng JPA schema generation trong lab

MỤC TIÊU: Để entity mapping và ddl-auto tạo/đồng bộ schema trong môi trường lab.

WARNING: `spring.jpa.hibernate.ddl-auto=update` tiện cho lab/dev nhưng không phải production migration strategy. Phần Advanced sẽ refactor sang Flyway/Liquibase.

6. ACTIVITY 1 – TẠO SPRING BOOT PROJECT VÀ DEPENDENCIES

Step 1.1 – Tạo Spring Initializr project

MỤC TIÊU: Có Spring Boot backend khởi động được trước khi thêm JWT.

- Spring Web
- Spring Data JPA
- Spring Security
- MS SQL Server Driver
- Validation (khuyến nghị bổ sung)
- Spring Boot Test

Step 1.2 – Kiểm tra `pom.xml`

MỤC TIÊU: Dependencies đủ cho Web, JPA, Security, SQL Server, test và JJWT.

Core dependencies

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<dependency>
  <groupId>com.microsoft.sqlserver</groupId>
  <artifactId>mssql-jdbc</artifactId>
  <scope>runtime</scope>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>

<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-test</artifactId>
  <scope>test</scope>
</dependency>
```

JJWT baseline theo Lab source

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.6</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>
```

Verification

```
mvn -q -DskipTests compile
mvn dependency:tree
```

PASS/FAIL: PASS nếu build thành công và không có dependency resolution error.

7. CẤU HÌNH SQL SERVER VÀ JWT PROPERTIES

Step 2.1 – `application.properties`

MỤC TIÊU: Backend kết nối SQL Server và có JWT secret/expiration từ configuration.

```
spring.application.name=lab06-jwt
server.port=8005
```

```
spring.datasource.url=jdbc:sqlserver://localhost:1433;databaseName=OrchidJWTLab;encrypt=true;trustServerCertificat
e=true
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASSWORD}
spring.datasource.driver-class-name=com.microsoft.sqlserver.jdbc.SQLServerDriver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Demo/course only: secret phải đủ mạnh và KHÔNG commit secret thật
security.jwt.secret-key=${JWT_SECRET}
security.jwt.expiration-time=3600000
```

SECURITY: Lab gốc minh họa secret trong properties. Bản hướng dẫn này chủ động refactor thành environment variable. Với HMAC, secret cần đủ entropy/key length tương ứng thuật toán; không dùng chuỗi ngắn dễ đoán.

Step 2.2 – Cấu hình Environment trong IntelliJ

MỤC TIÊU: Không hard-code secrets trong repository.

```
DB_USER=sa
DB_PASSWORD=<your-local-password>
JWT_SECRET=<base64-or-strong-secret-for-your-lab>
```

PASS/FAIL: PASS nếu application start và Hibernate kết nối database.

8. ACTIVITY 2 – ENTITY, DTO VÀ REPOSITORY

Step 3.1 – Tạo `User` entity implementing `UserDetails`

MỤC TIÊU: User vừa là JPA entity vừa cung cấp authentication details theo course baseline.

User.java

```
@Entity
@Table(name = "users")
public class User implements UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(nullable = false)
    private String fullName;

    @Column(nullable = false, unique = true, length = 100)
    private String email;

    @Column(nullable = false)
    private String password;

    @CreationTimestamp
    @Column(name = "created_at", updatable = false)
    private Date createdAt;

    @UpdateTimestamp
    @Column(name = "updated_at")
    private Date updatedAt;

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return List.of(); // Core Lab: chưa có role
    }

    @Override public String getUsername() { return email; }
    @Override public String getPassword() { return password; }

    // Với Spring Security hiện đại, các method account flags có default behavior
    // tùy version; nếu course skeleton yêu cầu override, trả true trong baseline.

    // getters/setters
}
```

NOTE: Core Lab giữ authorities rỗng đúng tinh thần tài liệu gốc. Role/authority được đưa sang Advanced Track để sinh viên hiểu rõ sự khác nhau giữa authentication và authorization.

Step 3.2 – Tạo request DTOs

MỤC TIÊU: Không nhận trực tiếp User entity làm request contract.

```
public class RegisterUserDto {
    @NotBlank
    @Email
    private String email;

    @NotBlank
    @Size(min = 8, max = 72)
    private String password;

    @NotBlank
    private String fullName;
    // getters/setters
}

public class LoginUserDto {
    @NotBlank @Email
    private String email;

    @NotBlank
    private String password;
    // getters/setters
}
```

Step 3.3 – Tạo `LoginResponse`

MỤC TIÊU: Response login có token và thời gian sống; không trả password/user entity thô.

```
public class LoginResponse {
    private String token;
    private long expiresIn;

    public LoginResponse(String token, long expiresIn) {
        this.token = token;
        this.expiresIn = expiresIn;
    }
    // getters/setters
}
```

Step 3.4 – Tạo `UserRepository`

MỤC TIÊU: Tra cứu User theo email phục vụ UserDetailsService/authentication.

```
@Repository
public interface UserRepository extends CrudRepository<User, Integer> {
    Optional<User> findByEmail(String email);
    boolean existsByEmail(String email);
}
```

PASS/FAIL: PASS nếu project compile và bảng `users` được tạo khi app start.

9. CẤU HÌNH AUTHENTICATION: USERDETAILSSERVICE, PASSWORDENCODER, PROVIDER

Step 4.1 – `ApplicationConfiguration`

MỤC TIÊU: Spring Security biết cách load User, hash password và authenticate username/password.

```
@Configuration
public class ApplicationConfiguration {

    private final UserRepository userRepository;

    public ApplicationConfiguration(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Bean
```

```
    UserDetailsService userDetailsService() {
        return username -> userRepository.findByEmail(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found"));
    }

    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    AuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
        provider.setUserDetailsService(userDetailsService());
        provider.setPasswordEncoder(passwordEncoder());
        return provider;
    }

    @Bean
    AuthenticationManager authenticationManager(
            AuthenticationConfiguration configuration) throws Exception {
        return configuration.getAuthenticationManager();
    }
}
```

SECURITY: Không bao giờ lưu password plaintext. Spring Security hiện khuyến nghị adaptive one-way functions như bcrypt, PBKDF2, scrypt hoặc Argon2; work factor phải được đo/tune theo hệ thống. Core Lab dùng BCrypt vì dễ triển khai và đúng course level.

10. JWT SERVICE – GENERATE, EXTRACT, VALIDATE

Step 5.1 – Tạo `JwtService`

MỤC TIÊU: Tách token concerns khỏi Controller và Filter.

JwtService.java

```
@Service
public class JwtService {

    @Value("${security.jwt.secret-key}")
    private String secretKey;

    @Value("${security.jwt.expiration-time}")
    private long jwtExpiration;

    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    public <T> T extractClaim(String token, Function<Claims, T> resolver) {
        return resolver.apply(extractAllClaims(token));
    }

    public String generateToken(UserDetails userDetails) {
        return generateToken(Map.of(), userDetails);
    }

    public String generateToken(
            Map<String, Object> extraClaims,
            UserDetails userDetails) {
        Instant now = Instant.now();
        return Jwts.builder()
            .claims(extraClaims)
            .subject(userDetails.getUsername())
            .issuedAt(Date.from(now))
            .expiration(Date.from(now.plusMillis(jwtExpiration)))
            .signWith(getSigningKey())
            .compact();
    }

    public boolean isTokenValid(String token, UserDetails userDetails) {
        String username = extractUsername(token);
        return username.equals(userDetails.getUsername())
```

```
                && !isTokenExpired(token);
    }

    public long getExpirationTime() {
        return jwtExpiration;
    }

    private boolean isTokenExpired(String token) {
        return extractExpiration(token).before(new Date());
    }

    private Date extractExpiration(String token) {
        return extractClaim(token, Claims::getExpiration);
    }

    private Claims extractAllClaims(String token) {
        return Jwts.parser()
            .verifyWith(getSigningKey())
            .build()
            .parseSignedClaims(token)
            .getPayload();
    }

    private SecretKey getSigningKey() {
        byte[] keyBytes = Decoders.BASE64.decode(secretKey);
        return Keys.hmacShaKeyFor(keyBytes);
    }
}
```

VERSION: JJWT API thay đổi giữa versions. Đoạn code trên dành cho course baseline 0.12.x; sinh viên phải compile theo đúng version trong `pom.xml`, không copy API từ blog dùng 0.11.x/0.9.x.

Step 5.2 – Kiểm tra JWT độc lập trước khi tạo Filter

MỤC TIÊU: Chứng minh generate → parse → validate hoạt động.

- Đăng ký một User hoặc tạo test UserDetails.
- Generate token.
- Extract subject/email.
- So sánh expiration.
- Tamper một ký tự token và quan sát parser reject.

PASS/FAIL: PASS nếu token hợp lệ parse được; token bị sửa phải fail validation/parsing.

11. JWT AUTHENTICATION FILTER

Step 6.1 – Tạo `JwtAuthenticationFilter` extends `OncePerRequestFilter`

MỤC TIÊU: Mỗi request Bearer token được validate và đưa Authentication vào SecurityContext.

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    public JwtAuthenticationFilter(
            JwtService jwtService,
            UserDetailsService userDetailsService) {
        this.jwtService = jwtService;
        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain filterChain)
            throws ServletException, IOException {

        String authHeader = request.getHeader(HttpHeaders.AUTHORIZATION);
```

```
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        String jwt = authHeader.substring(7);
        String userEmail;

        try {
            userEmail = jwtService.extractUsername(jwt);
        } catch (JwtException | IllegalArgumentException ex) {
            response.sendError(HttpServletResponse.SC_UNAUTHORIZED,
                               "Invalid token");
            return;
        }

        Authentication current =
            SecurityContextHolder.getContext().getAuthentication();

        if (userEmail != null && current == null) {
            UserDetails userDetails =
                userDetailsService.loadUserByUsername(userEmail);

            if (jwtService.isTokenValid(jwt, userDetails)) {
                UsernamePasswordAuthenticationToken authToken =
                    new UsernamePasswordAuthenticationToken(
                        userDetails, null, userDetails.getAuthorities());
                authToken.setDetails(
                    new WebAuthenticationDetailsSource().buildDetails(request));
                SecurityContextHolder.getContext()
                    .setAuthentication(authToken);
            }
        }

        filterChain.doFilter(request, response);
    }
}
```

CORE: Filter xác thực token, không quyết định business authorization. Authorization rules nằm trong SecurityFilterChain/method security.

12. SECURITYFILTERCHAIN – PUBLIC VS PROTECTED ENDPOINTS

Step 7.1 – Tạo `SecurityConfiguration`

MỤC TIÊU: Cho phép signup/login công khai; các endpoint khác cần authenticated user.

```
@Configuration
@EnableWebSecurity
public class SecurityConfiguration {

    private final AuthenticationProvider authenticationProvider;
    private final JwtAuthenticationFilter jwtAuthenticationFilter;

    public SecurityConfiguration(
            AuthenticationProvider authenticationProvider,
            JwtAuthenticationFilter jwtAuthenticationFilter) {
        this.authenticationProvider = authenticationProvider;
        this.jwtAuthenticationFilter = jwtAuthenticationFilter;
    }

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(sm -> sm
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/**").permitAll()
                .anyRequest().authenticated())
            .authenticationProvider(authenticationProvider)
            .addFilterBefore(
                jwtAuthenticationFilter,
```

```
                UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

SECURITY: `csrf.disable()` ở đây chỉ được dùng vì Core Lab là stateless REST API dùng Bearer token trong Authorization header và không dùng cookie-based browser session. Không biến đây thành quy tắc "REST là luôn tắt CSRF".

13. AUTHENTICATION SERVICE – SIGNUP VÀ LOGIN

Step 8.1 – `signup()`

MỤC TIÊU: Hash password trước khi save và chặn email duplicate.

```
@Service
public class AuthenticationService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final AuthenticationManager authenticationManager;

    public AuthenticationService(
            UserRepository userRepository,
            PasswordEncoder passwordEncoder,
            AuthenticationManager authenticationManager) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
        this.authenticationManager = authenticationManager;
    }

    public User signup(RegisterUserDto input) {
        if (userRepository.existsByEmail(input.getEmail())) {
            throw new IllegalStateException("Email already registered");
        }

        User user = new User();
        user.setFullName(input.getFullName());
        user.setEmail(input.getEmail());
        user.setPassword(passwordEncoder.encode(input.getPassword()));
        return userRepository.save(user);
    }

    public User authenticate(LoginUserDto input) {
        authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                input.getEmail(), input.getPassword()));

        return userRepository.findByEmail(input.getEmail())
            .orElseThrow();
    }
}
```

PASS/FAIL: PASS nếu DB password bắt đầu bằng BCrypt hash format và không bằng password gửi từ Postman.

14. AUTHENTICATION CONTROLLER

Step 9.1 – `/auth/signup` và `/auth/login`

MỤC TIÊU: Expose HTTP contract, không nhét authentication logic vào Controller.

```
@RestController
@RequestMapping("/auth")
public class AuthenticationController {

    private final JwtService jwtService;
    private final AuthenticationService authenticationService;

    public AuthenticationController(
            JwtService jwtService,
            AuthenticationService authenticationService) {
        this.jwtService = jwtService;
        this.authenticationService = authenticationService;
```

```
    }

    @PostMapping("/signup")
    public ResponseEntity<?> register(
            @Valid @RequestBody RegisterUserDto dto) {
        User user = authenticationService.signup(dto);
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(Map.of("id", user.getId(),
                         "email", user.getEmail(),
                         "fullName", user.getFullName()));
    }

    @PostMapping("/login")
    public ResponseEntity<LoginResponse> login(
            @Valid @RequestBody LoginUserDto dto) {
        User authenticated = authenticationService.authenticate(dto);
        String jwt = jwtService.generateToken(authenticated);
        return ResponseEntity.ok(
            new LoginResponse(jwt, jwtService.getExpirationTime()));
    }
}
```

15. USER SERVICE VÀ PROTECTED CONTROLLER

Step 10.1 – `UserService`

MỤC TIÊU: Tách user query khỏi Controller.

```
@Service
public class UserService {
    private final UserRepository repository;

    public UserService(UserRepository repository) {
        this.repository = repository;
    }

    public List<User> allUsers() {
        List<User> users = new ArrayList<>();
        repository.findAll().forEach(users::add);
        return users;
    }
}
```

Step 10.2 – `UserController`

MỤC TIÊU: Xác minh SecurityContext bằng ` /users/me` và protected list.

```
@RestController
@RequestMapping("/users")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/me")
    public ResponseEntity<?> authenticatedUser(Authentication authentication) {
        User current = (User) authentication.getPrincipal();
        return ResponseEntity.ok(Map.of(
            "id", current.getId(),
            "email", current.getEmail(),
            "fullName", current.getFullName()));
    }

    @GetMapping
    public ResponseEntity<List<Map<String, Object>>> allUsers() {
        var result = userService.allUsers().stream()
            .map(u -> Map.<String,Object>of(
                "id", u.getId(),
                "email", u.getEmail(),
                "fullName", u.getFullName()))
            .toList();
        return ResponseEntity.ok(result);
```

```
    }
}
```

SECURITY: Không serialize `User` entity nguyên trạng nếu entity có password field. Dùng response DTO hoặc explicit projection/map để tránh password hash bị lộ.

16. GLOBAL ERROR HANDLING – API RESPONSE CÓ Ý NGHĨA

Step 11.1 – Validation và duplicate errors

MỤC TIÊU: Client nhận status/message có cấu trúc.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    ResponseEntity<?> validation(MethodArgumentNotValidException ex) {
        Map<String, String> errors = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(e ->
            errors.put(e.getField(), e.getDefaultMessage()));
        return ResponseEntity.badRequest().body(Map.of(
            "error", "validation_failed",
            "fields", errors));
    }

    @ExceptionHandler(IllegalStateException.class)
    ResponseEntity<?> conflict(IllegalStateException ex) {
        return ResponseEntity.status(HttpStatus.CONFLICT)
            .body(Map.of("error", ex.getMessage()));
    }

    @ExceptionHandler(BadCredentialsException.class)
    ResponseEntity<?> badCredentials(BadCredentialsException ex) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
            .body(Map.of("error", "invalid_credentials"));
    }
}
```

NOTE: Authentication exceptions có thể bị xử lý trong Security filter chain trước khi vào MVC advice. Phần Advanced sẽ refactor `AuthenticationEntryPoint` / `AccessDeniedHandler` để chuẩn hóa 401/403.

17. CHẠY BACKEND VÀ KIỂM TRA DATABASE

Step 12.1 – Run project

MỤC TIÊU: Backend listen ở port 8005.

```
mvn spring-boot:run
# hoặc chạy Application class trong IntelliJ
```

PASS/FAIL: PASS nếu log báo application started và không có datasource/security bean cycle error.

Step 12.2 – Kiểm tra bảng users

MỤC TIÊU: Schema và persistence đúng.

```
USE OrchidJWTLab;
SELECT TOP 20 id, full_name, email, password, created_at, updated_at
FROM users
ORDER BY id DESC;
```

SECURITY: Chỉ dùng query password hash để học; không screenshot hoặc phát tán credential/hash thật của hệ thống production.

18. KIỂM THỬ POSTMAN – CORE LAB

Step 13.1 – Tạo Postman Environment

MỤC TIÊU: Giảm copy/paste token và base URL.

Variable	Initial/Current
baseUrl	http://localhost:8005

Variable	Initial/Current
token	(để trống, set sau login)

Step 13.2 – POST `/auth/signup`

MỤC TIÊU: Đăng ký User mới.

```
POST {{baseUrl}}/auth/signup
Content-Type: application/json

{
  "fullName": "SBA301 Student",
  "email": "student01@fpt.edu.vn",
  "password": "P@ssword123"
}
```

PASS/FAIL: PASS: 201 Created; DB có row; password DB khác plaintext.

Step 13.3 – POST `/auth/login`

MỤC TIÊU: Credential đúng trả JWT.

```
POST {{baseUrl}}/auth/login
Content-Type: application/json

{
  "email": "student01@fpt.edu.vn",
  "password": "P@ssword123"
}
```

PASS/FAIL: PASS: 200 OK + `token` + `expiresIn`.

Step 13.4 – Tự động lưu token trong Postman

MỤC TIÊU: Login xong các request protected dùng token tự động.

Postman Tests tab

```
const json = pm.response.json();
pm.environment.set("token", json.token);
```

Step 13.5 – GET `/users/me`

MỤC TIÊU: Xác minh token -> authenticated principal.

```
GET {{baseUrl}}/users/me
Authorization: Bearer {{token}}
```

PASS/FAIL: PASS: 200 OK và email đúng user đã login.

Step 13.6 – GET `/users`

MỤC TIÊU: Xác minh endpoint protected.

```
GET {{baseUrl}}/users
Authorization: Bearer {{token}}
```

PASS/FAIL: PASS: 200 OK; response không chứa `password`.

19. NEGATIVE TEST MATRIX – BẮT BUỘC

ID	Case	Request/Setup	Expected
N01	No token	GET /users/me, no Authorization	401
N02	Wrong scheme	Authorization: Basic xyz	401
N03	Malformed token	Bearer abc.def	401
N04	Tampered signature	change 1 char in JWT	401
N05	Expired token	very short TTL / old token	401
N06	Wrong password	POST /auth/login	401
N07	Unknown email	POST /auth/login	401
N08	Duplicate email	POST /auth/signup existing email	409
N09	Invalid email	signup invalid format	400
N10	Weak/short password	signup < min length	400

ID	Case	Request/Setup	Expected
N11	Public signup	POST /auth/signup no token	201/400 based payload
N12	Public login	POST /auth/login no token	200/401

20. TRACE END-TO-END MỘT REQUEST

20.1. Login trace

```
POST /auth/login
 -> AuthenticationController.login()
 -> AuthenticationService.authenticate()
 -> AuthenticationManager.authenticate()
 -> ProviderManager
 -> DaoAuthenticationProvider
 -> UserDetailsService.loadUserByUsername(email)
 -> UserRepository.findByEmail()
 -> PasswordEncoder.matches(raw, encoded)
 <- authenticated User
 -> JwtService.generateToken(User)
 <- LoginResponse(token, expiresIn)
```

20.2. Protected request trace

```
GET /users/me + Bearer JWT
 -> SecurityFilterChain
 -> JwtAuthenticationFilter
 -> parse/verify JWT
 -> UserDetailsService.loadUserByUsername(subject)
 -> JwtService.isTokenValid()
 -> SecurityContextHolder.setAuthentication(...)
 -> authorization: authenticated()
 -> UserController.authenticatedUser(Authentication)
 <- 200 current user
```

21. DEBUGGING – LỖI THƯỜNG GẶP VÀ CÁCH XỬ LÝ

Triệu chứng	Nguyên nhân thường gặp	Cách kiểm tra
401 dù token mới	secret mismatch / parse lỗi / Bearer header sai	breakpoint filter + JwtService
403 thay vì 401	đã authenticated nhưng thiếu authority/policy	SecurityContext + rules
Password login luôn fail	password DB plaintext/hash khác encoder	DB value + encoder.matches
Bean cycle/start fail	dependency graph security config	startup stack trace
JJWT compile error	API/version mismatch	pom + JJWT version
Token invalid key length	HMAC secret quá ngắn/không decode đúng	secret format/key bytes
User response lộ password	serialize entity trực tiếp	response DTO review
Postman OK, browser fail	CORS/preflight	DevTools Network
500 cho bad token	Filter exception không map 401	filter/entry point
Duplicate signup race	chỉ pre-check existsByEmail	DB UNIQUE constraint

21.1. Debugging decision tree

```
Protected endpoint failed
|
+-- Authorization header missing? -> expected 401
|
+-- Header starts with "Bearer "?
|     +-- no -> 401
|
+-- JWT parse/signature/expiration valid?
|     +-- no -> 401
|
+-- subject/email found in DB?
|     +-- no -> 401
|
+-- SecurityContext contains Authentication?
|     +-- no -> inspect filter ordering
|
+-- Endpoint requires role/authority?
```

```
+-- missing authority -> 403
+-- enough authority -> controller executes
```

22. BREAK-IT → DIAGNOSE → FIX WORKSHOP

Break-It	Expected failure	Diagnosis target	Fix
Remove PasswordEncoder in provider	login fails	DaoAuthenticationProvider	restore matching encoder
Store raw password	credential/security defect	DB password	always encode at signup
Change JWT secret after login	old token rejected	signature verification	expected invalidation/key strategy
Put JWT filter after username filter	unexpected flow	filter order	addFilterBefore
Return User entity directly	password hash visible	serialization	response DTO
Permit `/users/**`	protected data public	authorization rules	restore authenticated/role
Set TTL 1 second	token expires quickly	exp claim	correct TTL/test expiration
Concatenate credential into logs	secret leakage	logging	redact/remove

23. AUTOMATED SECURITY TESTS – COURSE-READY

Step 23.1 – MockMvc public/protected contract

MỤC TIÊU: Không phụ thuộc hoàn toàn vào Postman.

```
@SpringBootTest
@AutoConfigureMockMvc
class SecurityContractTest {

    @Autowired MockMvc mvc;

    @Test
    void protectedEndpointWithoutToken_isUnauthorized() throws Exception {
        mvc.perform(get("/users/me"))
           .andExpect(status().isUnauthorized());
    }

    @Test
    void signupValidationRejectsBadEmail() throws Exception {
        mvc.perform(post("/auth/signup")
                .contentType(MediaType.APPLICATION_JSON)
                .content("""
                    {"fullName":"A","email":"bad","password":"12345678"}
                """))
           .andExpect(status().isBadRequest());
    }
}
```

Step 23.2 – Integration test login → JWT → protected endpoint

MỤC TIÊU: Chứng minh full security pipeline bằng test.

- Create test user qua repository/service với encoded password.
- POST `/auth/login` và parse token.
- GET `/users/me` với `Bearer <token>`.
- Assert 200 + correct email.
- Tamper token rồi assert 401.

24. CORE LAB INTEGRATION GATE G1–G14

Gate	Evidence
G1 Build	mvn test/compile pass
G2 DB	SQL Server connection + users table
G3 Password	stored password hashed
G4 Signup	201 valid registration
G5 Login	200 + JWT
G6 JWT	subject/exp/signature valid
G7 Filter	Bearer token populates SecurityContext

Gate	Evidence
G8 Public paths	/auth/** accessible without token
G9 Protected path	/users/me requires token
G10 No token	401
G11 Tampered/expired	401
G12 DTO safety	response never exposes password
G13 Automated tests	minimum happy + negative
G14 Explain	401 vs 403; authentication vs authorization

25. CHECKLIST HOÀN THÀNH LAB 06

☐ 1. Project Spring Boot + dependencies build được.

☐ 2. Datasource SQL Server hoạt động.

☐ 3. JWT secret không hard-code vào Git.

☐ 4. User implements UserDetails theo course baseline.

☐ 5. Password được BCrypt encode.

☐ 6. UserRepository findByEmail/existsByEmail hoạt động.

☐ 7. JwtService generate/parse/validate đúng.

☐ 8. JwtAuthenticationFilter đọc Bearer token đúng.

☐ 9. SecurityFilterChain stateless và phân public/protected đúng.

☐ 10. /auth/signup chạy.

☐ 11. /auth/login trả JWT.

☐ 12. /users/me với JWT chạy.

☐ 13. /users/me không JWT bị từ chối.

☐ 14. Tampered/expired token bị từ chối.

☐ 15. Không response password hash.

☐ 16. Postman Collection có token variable.

☐ 17. Có negative-test evidence.

☐ 18. Có automated test tối thiểu.

26. BỘ BẰNG CHỨNG KHUYẾN NGHỊ SINH VIÊN NỘP

Evidence	Nội dung
E1	Project structure + pom.xml dependencies
E2	application config đã che secrets
E3	DB users table + BCrypt hash
E4	Signup 201
E5	Login 200 + token (che phần token khi báo cáo công khai)
E6	/users/me 200 với token
E7	No-token 401
E8	Tampered/expired 401
E9	Automated test report
E10	Một Break-It → Diagnose → Fix note

27. QUICK RUN CARD – CHẠY LẠI PROJECT SAU KHI ĐÃ HOÀN THÀNH

```
# 1. Start SQL Server
# 2. Set environment variables
DB_USER=...
DB_PASSWORD=...
JWT_SECRET=...
```

```
# 3. Run tests
mvn test

# 4. Run app
mvn spring-boot:run

# 5. Postman
POST http://localhost:8005/auth/login
GET  http://localhost:8005/users/me
Authorization: Bearer <token>
```

28. ADVANCED EXTENSION & REFACTOR TRACK – TỰ NGHIÊN CỨU NÂNG CAO

ADVANCED: Các track dưới đây KHÔNG phải điều kiện để pass Core Lab. Sinh viên chọn theo mức độ. Mỗi track phải nộp: mục tiêu → thiết kế → code diff → test evidence → threat/trade-off note. Không chấp nhận "copy code chạy được" mà không giải thích security semantics.

28.1. Track A – Role/Authority và Authorization thật sự

Core Lab chỉ authentication với authorities rỗng. Mở rộng User với `Role`/`GrantedAuthority`, ví dụ USER và ADMIN; `/users/me` cho authenticated user, `/users` chỉ ADMIN.

```
public enum Role { USER, ADMIN }

@Enumerated(EnumType.STRING)
@Column(nullable = false)
private Role role = Role.USER;

@Override
public Collection<? extends GrantedAuthority> getAuthorities() {
    return List.of(new SimpleGrantedAuthority("ROLE_" + role.name()));
}

// Security rule
.requestMatchers(HttpMethod.GET, "/users").hasRole("ADMIN")
.anyRequest().authenticated()
```

- Test USER → `/users` = 403.
- Test ADMIN → `/users` = 200.
- Giải thích 401 khác 403.
- Không cho client signup tự chọn ADMIN.

28.2. Track B – Chuẩn hóa 401/403 với AuthenticationEntryPoint & AccessDeniedHandler

Refactor error responses để authentication failure và authorization failure có JSON contract thống nhất.

```
{
  "timestamp": "...",
  "status": 401,
  "error": "unauthorized",
  "path": "/users/me"
}

{
  "timestamp": "...",
  "status": 403,
  "error": "forbidden",
  "path": "/users"
}
```

28.3. Track C – Refresh Token + Rotation

Thêm access token TTL ngắn và refresh token dài hơn. Refresh token phải có persistence/revocation state hoặc chiến lược tương đương; không chỉ tạo một JWT thứ hai rồi gọi là refresh token.

- POST `/auth/refresh` nhận refresh token và cấp access token mới.
- Rotate refresh token: token cũ bị invalidated sau khi dùng.
- Lưu token family / revokedAt / expiresAt hoặc hashed refresh token.

- Test reuse của refresh token cũ.
- Không đưa refresh token vào URL/query string.

SECURITY: OAuth 2.0 Security BCP RFC 9700 yêu cầu public clients dùng sender-constrained refresh tokens hoặc refresh-token rotation để phát hiện replay. Đây là hướng nghiên cứu, không biến Core Lab custom JWT thành OAuth server.

28.4. Track D – Logout / Token Revocation

JWT access token stateless khó revoke trước expiration. Sinh viên nghiên cứu các chiến lược: short TTL + refresh revocation; denylist theo `jti`; user/token version; centralized session/token store.

Chiến lược	Ưu	Nhược
Short access TTL	simple/stateless	window vẫn tồn tại
Refresh token revoke	phổ biến	access token hiện tại còn sống
JWT denylist by jti	revoke ngay	state/cache lookup
Token version per user	revoke all user tokens	DB/cache lookup/version design

28.5. Track E – JWT Claims Governance: iss / aud / jti / roles

- Thêm `iss` issuer và validate.
- Thêm `aud` audience và validate đúng API.
- Thêm `jti` unique token id phục vụ audit/revocation.
- Đưa role/scope vào claim nếu architecture yêu cầu, nhưng xác định source-of-truth/staleness.
- Không nhét dữ liệu nhạy cảm vào JWT payload.

28.6. Track F – Refactor HMAC secret → RSA/ECDSA + JWK

Thay shared secret bằng asymmetric signing: authorization side giữ private key; resource/API chỉ cần public key để verify. Nghiên cứu key rotation và JWKS.

- Generate dev key pair an toàn.
- Sign bằng private key, verify bằng public key.
- Thiết kế `kid` + multiple active verification keys.
- Không commit private key.

28.7. Track G – Refactor custom filter → Spring Security OAuth2 Resource Server

Đây là refactor quan trọng để hiểu production direction. Spring Security hiện hỗ trợ Resource Server cho JWT và opaque bearer tokens; với JWT, framework có thể validate signature/`exp`/`nbf`/`iss` và map scopes/authorities theo configuration.

- Tách việc "phát token" khỏi API resource server.
- Config `spring-security-oauth2-resource-server` + issuer/jwk-set-uri hoặc JwtDecoder phù hợp.
- Loại bỏ custom JwtAuthenticationFilter khi framework đảm nhiệm bearer authentication.
- So sánh code size, error handling, key rotation, standards support.
- Viết migration note: custom-JJWT lab → standards-based Resource Server.

28.8. Track H – Password Security Refactor

- Benchmark BCrypt work factor trên máy local.
- So sánh BCrypt với Argon2/PBKDF2/scrypt theo Spring Security docs.
- Implement `DelegatingPasswordEncoder` để hỗ trợ algorithm migration.
- Đo login latency và ghi hardware/JDK/version.
- Không benchmark bằng production passwords.

28.9. Track I – DTO / Domain Refactor

Tách `User` JPA entity khỏi `UserDetails` bằng custom principal (`SecurityUser`/adapter), tránh domain entity phụ thuộc trực tiếp Spring Security.

```
UserEntity (JPA/domain persistence)
      | mapped by
      v
SecurityUser implements UserDetails
      ^
      | created by
CustomUserDetailsService
```

- Response DTO tuyệt đối không có password.
- Request DTO có validation.
- Mapping entity↔DTO rõ ràng.
- Service không nhận mọi field từ client một cách mass-assignment.

28.10. Track J – Method Security

Bổ sung `@EnableMethodSecurity` và `@PreAuthorize` cho use case cần defense-in-depth; so sánh URL rules với method rules.

```
@EnableMethodSecurity

@PreAuthorize("hasRole('ADMIN')")
public List<UserSummaryDto> allUsers() { ... }
```

28.11. Track K – CORS + React Integration

Kết nối React/Vite từ Lab 05. Cho phép đúng origin/method/header cần thiết; không dùng wildcard tùy tiện khi credentials/cookies xuất hiện.

- React login lưu token theo strategy đã phân tích.
- Axios interceptor gắn `Authorization: Bearer`.
- 401 → clear auth state / redirect login.
- 403 → thông báo insufficient permission.
- Kiểm OPTIONS preflight riêng với auth request.

28.12. Track L – Rate Limiting / Brute-force Defense

- Giới hạn login attempts theo account/IP/risk signals.
- Backoff/lockout policy tránh DoS self-lock.
- Audit failed login mà không log password.
- Test rate limit response 429.
- Thảo luận app-level vs reverse proxy/API gateway implementation.

28.13. Track M – Account Lifecycle

- Email verification / activation.
- Password reset token: single-use + expiry + hashing.
- Account disabled/locked flags đưa vào UserDetails.
- Password changedAt/tokenVersion để invalidate sessions theo policy.
- Audit security-sensitive changes.

28.14. Track N – API Contract & Problem Details

Refactor lỗi theo RFC-style Problem Details (`application/problem+json`) hoặc một contract thống nhất. Tách validation, authentication, authorization, conflict và internal errors.

28.15. Track O – Security Test Suite mở rộng

Test family	Cases
Authentication	valid login, bad password, unknown user
JWT integrity	malformed, tampered, wrong key, expired
Authorization	USER vs ADMIN
Input	invalid email, weak password, duplicate
CORS	allowed/disallowed origin/preflight
Refresh	rotation/reuse/revocation
Regression	public endpoint accidentally protected or protected endpoint accidentally public

28.16. Track P – Observability & Audit

- Log authentication success/failure events mà không log secrets/token.
- Correlation/request ID.
- Metrics: login failure rate, 401/403 count, auth latency.
- Audit admin role change/token revocation.
- Redact Authorization header ở logs/APM.

28.17. Track Q – Database & Migration Refactor

- Thay `ddl-auto=update` bằng Flyway hoặc Liquibase.
- Add migration cho users.role / refresh_tokens / token_version.
- Đặt UNIQUE constraint email trong migration.
- Test clean DB migration từ V1 → latest.
- Dùng `ddl-auto=validate` ở production-like profile.

28.18. Track R – Profiles, Secrets & Deployment

- Tạo `application-dev.properties`, `application-test.properties` và production env config.
- Dùng environment variables/secret manager thay secrets trong repo.
- Containerize backend + SQL Server dev stack nếu course cho phép.
- Health/readiness checks không leak configuration.
- HTTPS/TLS là bắt buộc khi truyền bearer token ngoài localhost/dev.

29. ADVANCED TRACK – MỨC ĐỘ VÀ RUBRIC TỰ NGHIÊN CỨU

Mức	Yêu cầu tối thiểu	Ví dụ
A – Foundation+	1 extension + tests	Role/ADMIN + 401/403 matrix
B – Advanced	2–3 extensions + threat note	Role + Refresh + Revocation
C – Refactor	Architecture refactor + comparison	Resource Server / Security principal separation
D – Research	Prototype + benchmark/standards evidence	Argon2 benchmark / JWK rotation / DPoP literature

29.1. Mẫu báo cáo self-study

- Problem / Threat: vấn đề gì cần giải quyết?
- Baseline: Core Lab hiện xử lý thế nào?
- Design: kiến trúc/refactor mới.
- Implementation: files/classes/config thay đổi.
- Positive tests + negative tests.
- Security trade-offs / known limitations.
- Official sources + version/date.
- Conclusion: MERGE / SHOULD EXPLORE / WATCH LATER.

30. NGUỒN VÀ PHẠM VI ÁP DỤNG

- Lab 06 gốc – Secure REST Services with Spring Boot using JWT: JWT introduction, Spring Boot 3 + SQL Server + JPA, User/UserDetails, DTOs, UserRepository, JwtService, Security configuration/filter, AuthenticationService/Controller, secured endpoints and Postman tests.
- Course Chapter 16 – Securing REST Services with Spring Boot: authentication/authorization, JWT and Spring Security architecture.
- Spring Security official reference (current check 2026-10-12): latest stable docs list Spring Security 7.1.1; architecture centers on SecurityContextHolder, AuthenticationManager/Provider and GrantedAuthority.
- Spring Security official password-storage guidance: adaptive one-way functions include bcrypt, PBKDF2, scrypt and Argon2; work factor should be tuned to the system.
- Spring Security official Resource Server docs: current framework supports JWT and opaque bearer tokens; JWT resource server validates bearer tokens and standard claims with issuer/JWK configuration.
- RFC 9700 (January 2025) – OAuth 2.0 Security Best Current Practice; used only for advanced refresh-token/security research, not as a claim that this lab implements OAuth 2.0.

IMPORTANT: Course Baseline và Current Production Direction phải được giữ tách biệt: hoàn thành Core Lab theo version/course contract trước; chỉ nâng cấp/refactor khi có branch riêng và regression tests.